

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

Институт (факультет)
Кафедра

Информационных технологий
Математическое и программное обеспечение ЭВМ

КУРСОВАЯ РАБОТА

по дисциплине

Программирование на ассемблере

на тему

Программирование на языке низкого уровня

Выполнил студент группы 1ПИБ-02-3оп-23

группа

направления подготовки (специальности)

09.03.04 Программная инженерия

шифр, наименование

Бекошин Никита Вячеславович

фамилия, имя, отчество

Руководитель

Виноградова Людмила Николаевна

фамилия, имя, отчество

доцент, кандидат наук

должность

Дата представления работы

« ____ » _____ 20 ____ г.

Заключение о допуске к защите

Оценка _____, _____

количество баллов

Подпись преподавателя _____

Череповец, _____

год

Содержание

Введение	3
Описание предметной области	5
Постановка задачи.....	7
Логическое проектирование.....	10
Физическое проектирование	14
Кодирование	16
Тестирование программы.....	22
Заключение	26
Список литературы	27
Приложение 1. Техническое задание	28
Приложение 2. Руководство пользователя	40
Приложение 3. Текст программы	43
Приложение 4. Листинг программы	44

Введение

Язык ассемблера, как язык программирования низкого уровня, предоставляет разработчику беспрецедентный контроль над ресурсами компьютера, что делает его незаменимым инструментом для достижения максимальной производительности и эффективного использования памяти. В отличие от языков высокого уровня, где абстракции, такие как операторы сложения, умножения или деления, скрывают детали реализации, в ассемблере программист напрямую управляет регистрами процессора, указателями памяти и флагами состояния. Это позволяет избежать накладных расходов, связанных с абстракциями, и минимизировать объем используемой памяти, что, в конечном итоге, способствует увеличению скорости выполнения программы.

Работа с ассемблером требует глубокого понимания архитектуры процессора. Вместо высокоуровневых операций, программист манипулирует регистрами, использует значения флагов (которые могут принимать значения только нуля или единицы) для контроля потока выполнения и отладки. Анализ флагов позволяет точно отслеживать поведение программы на уровне машинного кода, что обеспечивает более тонкую и эффективную отладку.

В данной курсовой работе особое внимание уделяется демонстрации возможностей параллельного выполнения операций на низком уровне. Параллелизм, реализованный на уровне машинного кода, где порядок выполнения некоторых операций не влияет на результат, позволяет максимально использовать ресурсы процессора, выполняя независимые операции одновременно. Это особенно актуально для задач, требующих высокой производительности, и подчеркивает преимущества применения низкоуровневых языков, таких как ассемблер, для достижения максимальной эффективности вычислений. Разработка программы, демонстрирующей параллельное выполнение, позволит практически оценить потенциал низкоуровневого программирования в оптимизации производительности.

Программист обращается к ассемблеру с целью эффективного использования памяти для работы программы, а значит и глубокой оптимизации кода, так как этот язык программирования низкого уровня, что подразумевает отсутствие абстракций, например, знак суммы, умножения, произведения и деления, они помогают производить расчёты без обращения к памяти. Абстракции, таким образом, могут дополнительно занимать память, что может приводить к относительному снижению производительности персонального компьютера, то есть к эффективному распределению ресурсов при работе написанной программы на языке машинных кодов. Вместо абстракций, программист работает с указателями или регистрами, он также использует значения флагов, которые могут принимать значения только нуля или единицы в процессе отладки, и он, в зависимости оттого, какое конкретное значение принимает определенный флаг, делает выводы, и таким образом отслеживает правильность или неправильность написания программы на ассемблере.

Описание предметной области

Предметной областью курсовой работы является системное программирование [1].

Системное программирование - это раздел программирования, в котором сочетаются исследования новых архитектур, алгоритмов, структур данных и др. и деятельность по проектированию, разработке, тестированию и сопровождению (поддержке) системного программного обеспечения (системного ПО), т. е. для создания новых информационных технологий.

Системное ПО является фундаментом, на котором базируется всё программное обеспечение (ПО) компьютеров. Различают системное ПО машинно-зависимое (предназначено для использования в семействах компьютеров с одной и той же системой команд) и переносимое, которое используется на компьютерах с разной архитектурой. Системное ПО применяют для управления ПО компьютеров и сетевыми коммуникациями, а также для поддержки выполнения прикладных программ. К системному ПО относятся операционные системы (ОС), программные средства организации компьютерных сетей и управления ими, системы управления базами данных (СУБД), средства промежуточного ПО (предоставляют выделенному классу приложений набор услуг, напрямую не предоставляемых ОС), инструментальные средства разработки и анализа программ, поддержки информационной безопасности и др. При разработке системного ПО используются методы программной инженерии; особое внимание уделяется качеству кода (включает минимизацию числа ошибок, простоту понимания и сопровождения, хорошую документированность и т. п.), надёжности и безопасности программ.

Системное программирование появилось в 1950-х гг., когда были созданы первые ОС, ассемблеры и компиляторы для мейнфреймов. Важным

этапом стало появление системного ПО, создаваемого некоммерческими сообществами системных программистов и распространяемого вместе с текстами программ (ОС FreeBSD и Linux, СУБД PostgreSQL и MySQL и др.), что позволило многочисленным пользователям освободиться от зависимости от производителей коммерческого системного ПО.

Постановка задачи

Требуется написать программу, которая вычисляет значение выражения под номером 6, и проверить работу программы для нескольких пар значений параметров N и j , а в процессе проверки программы в отладчике для выполнения в пошаговом режиме подпрограммы обработки прерывания необходимо команду INT выполнить в режиме пошагового выполнения команды (нажатием клавиш Alt+F7).

6. Вычислить значение произведения: $\prod_{i=1}^N i^3 \cdot j$

Подпрограмма должна:

- выполнять действия, указанные в конкретном задании (под номером 6), при этом подпрограмме должны передаваться параметры N и j ;
- выполняться через вызов пользовательского прерывания (например, INT 60h). Адрес подпрограммы должен быть занесен в таблицу векторов прерываний при помощи функций DOS 25h и 35h;
- принимать значения параметров N и j только при условии, если эти значения нетривиальны, то есть ни один из двух параметров не содержит ноль или единицу в качестве значения. Значение параметра N должно быть больше единицы;

Подпрограмма должна возвращать результаты работы в регистрах общего назначения. После вызова подпрограммы программа должна восстановить адрес старого обработчика прерывания при помощи тех же функций DOS (25h и 35h). Подпрограмма должна принимать параметры N и j , как только они будут передаваться в неё. Параметры N и j могут передаваться в подпрограмму обработки прерывания через регистры общего назначения или через ячейки памяти;

Подпрограмма должна учитывать ситуацию переполнения разрядной сетки для факториальных выражений при вычислении произведений значений N и j ;

Подпрограмма должна учитывать приоритеты арифметических операций при вычислении значений сумм и произведений с использованием арифметических операций;

Подпрограмма должна выполнять команду INT в режиме пошагового выполнения команды (выполнение активируется нажатием комбинации клавиш Alt+F7), чтобы подпрограмма могла выполнять обработку прерывания в процессе проверки работы программы в отладчике.

Программа должна выполнять требования к вызову подпрограммы, то есть включать в себя возможность автоматически восстанавливать адрес старого обработчика прерывания при помощи функций DOS (25h и 35h).

Подпрограмма должна выполнять требования к параметрам, то есть включать в себя возможность проводить обработку прерывания и принимать параметры N и j как только они будут передаваться в неё через ячейки памяти.

Подпрограмма должна выполнять требования учета ситуации переполнения разрядной сетки для факториальных выражений в случаях вычислений произведений значений N и j.

Подпрограмма должна выполнять требования к учету приоритетов арифметических операций (умножение, деление, вычитание, сложение) при вычислении значений сумм и произведений с использованием этих операций, в том числе операции возведения в степень «^», в случае её неявного указания, она выполняется только для значения, перед которым стоит эта операция, а значение, которое стоит после операции «^», является показателем степени, если правее от показателя степени имеются еще арифметические операции с другими значениями, то они не относятся к показателю степени значения, к которому применяется операция «^». Явное указание к арифметической операции «^» к более чем одному операнду осуществляется за счет скобок «(» и «)», которые означают левую и правую границы проведения арифметического расчета значений операндов, между которыми установлены арифметические операции. Эти скобки допустимо использовать как в основании степени, так и в её показателе. Стоит отметить, что операции умножения и деления имеют

одинаковый приоритет, однако эти операции выполняются по отношению к операциям «+» и «-» в первую очередь, то есть они приоритетнее последних двух.

Требования к проведению обработки прерывания в реальном времени включают в себя возможность подпрограммы выполнять команду INT в режиме пошагового выполнения команды.

Логическое проектирование

Описание программы:

Основные шаги программы:

1. Инициализация:

Программа данных:

N: Целое неотрицательное число, которое перед вызовом подпрограммы заносится из ячейки памяти, связанной с параметром N, в регистр AX, чтобы проверить это число на наличие знака, если условный переход с помощью команды [2] JS (SF=1) на метку «error_main» совершается, то это приводит к выводу ошибки для предупреждения Пользователя изменить параметр значений N на допустимый, чтобы затем повторить запуск программы снова. Однако, если в результате использования команды CMP (сравнение с нулем как второго операнда), которая вычитает второй операнд из первого, не совершился переход на метку ошибки с названием «error_main», то команда CMP теперь сравнивает значение параметра N с единицей, если значение регистра AX больше единицы, то совершается условный переход (на метку «NEXT». Если значение регистра AX окажется равным единице, то произойдет условный переход на метку «error_main» по команде JE (ZF=1), дальше происходит сравнение значения регистра AX с нулем (повторно), чтобы совершить условный переход на метку ошибки, если AX=0 (JE проверит флаг ZF, установится ли он в единицу после вычитания нуля из значения регистра AX).

Программа, которая выполняется по адресу метки с названием «NEXT», здесь будет проверяться значение параметра j, оно будет загружено в регистр AX. Происходит сравнение с единицей, если AX окажется равным единице, если AX=1, то происходит условный переход (JE) на метку «error_main», затем происходит сравнение с нулем, выполняется команда условного перехода (JE), чтобы передать управление метке «error_main».

j: Целое число.

result: Переменная для хранения конечного результата.

error_message: Сообщение об ошибке, если входные данные некорректны.

overflow_message: Сообщение об ошибке при переполнении.

2. Установка обработчика прерывания:

Адрес подпрограммы ISR (Interrupt Service Routine) устанавливается как обработчик прерывания INT 60h.

Для этого используется функция INT 21h с кодом 25h.

Старый обработчик прерывания сохраняется в OLD_ISR.

4. Вызов прерывания:

Вызывается прерывание INT 60h, что перенаправляет управление к подпрограмме ISR.

5. Восстановление старого обработчика прерывания:

После выполнения ISR старый обработчик прерывания, который был сохранен в OLD_ISR, восстанавливается. В регистр AH загружается значение 35h. Это номер функции DOS, которая позволяет получить текущий вектор прерывания (адрес обработчика).

6. Завершение программы:

Программа завершает свою работу.

Если входные данные некорректны, то выводится сообщение об ошибке и программа завершается.

Если произойдет переполнение, то выводится сообщение о переполнении и результат обнуляется.

Описание подпрограммы:

Обработчик прерывания (ISR): описание подпрограммы ISR (Interrupt Service Routine)

Подпрограмма ISR (Interrupt Service Routine) предназначена для обработки прерывания INT 60h и выполнения основного вычисления.

Основные шаги подпрограммы ISR:

1. Копирование N и j в регистры:

- Значение N копируется в регистр AX.
- Значение j копируется в регистр BX.

2. Инициализация:

Значение N копируется в регистр CX, который используется как счетчик цикла.

Регистр SI инициализируется значением единицы (используется как счетчик i).

Регистр BP инициализируется значением единицы (используется для хранения промежуточного результата).

Регистр DI присваивается значение BX (j).

3. Цикл вычислений:

Цикл loop_i: Выполняется N раз.

В каждой итерации:

Регистр AX присваивается значение SI (текущее значение i).

Вычисляется $i^3 * j$ путем последовательных умножений и сохраняется в регистре AX.

Текущее значение результата (в регистре BP) умножается на (i^3*j) с сохранением результата в регистрах DX:AX.

Проверяется переполнение. Если переполнение произошло (бит переноса CF = 1), то управление передается на метку «overflow».

Результат умножения копируется в BP.

Значение i (в регистре SI) увеличивается на единицу.

Счетчик цикла CX уменьшается на единицу. Если CX не равен нулю, происходит переход к началу цикла.

4. Сохранение результата:

После завершения цикла, младшая часть результата (хранящаяся в регистре BP) копируется в регистр AX и сохраняется в память по адресу переменной result.

Выполняется переход на метку «end_isr».

5. Обработка переполнения:

Если во время вычислений произошло переполнение (бит переноса CF установлен), то выполняется переход на метку «overflow»:

Выводится сообщение об ошибке `overflow_message`.

Значение результата (AX) обнуляется.

Выполняется переход на метку «`end_isr`».

6. Возврат из прерывания:

Команда IRET (Interrupt Return) завершает выполнение подпрограммы прерывания и возвращает управление в основную программу.

7. Обработчик ошибок:

Метка «`error`» используется для обработки ситуаций, когда входные данные N или j некорректны, однако она не используется в коде программы (так как проверка выполняется в основном коде).

Таким образом, программа выполняет вычисление произведений выражения значения параметра i в кубе, умноженного на значение параметра j , и обрабатывает возможные ошибки (некорректные входные данные и переполнение). Подпрограмма ISR реализует основную логику вычислений.

Физическое проектирование

В табл. 1 содержатся все переменные, которые использовались в процессе работы программы:

Табл. 1

Переменные

Наименование	Обозначение	Тип данных
Ячейка памяти, которая хранит значение параметра N, перед тем, как запустить программу, Пользователь инициализирует эту ячейку памяти значением целочисленного типа, то есть оно должно быть нетривиально, а также больше нуля, иначе программа не запустит подпрограмму, предупредив пользователя об ошибке с выводением соответствующего сообщения на экран.	N	DW (define word)
Ячейка памяти, которая хранит значение параметра j, перед тем, как запустить программу, Пользователь инициализирует эту ячейку памяти значением целочисленного типа, оно должно быть нетривиально, но может быть отрицательным, иначе программа не запустит подпрограмму, предупредив пользователя об ошибке с выводением соответствующего сообщения на экран.	j	DW (define word)
Ячейка памяти, которая хранит итоговое значение вычисленного подпрограммой произведения выражений, подпрограмма, перед тем как завершить свою работу, возвращает вычисленное в регистрах общего назначения значение путем загрузки результата вычисления в ячейку памяти с именем result. После работы	result	DW (define word)

программы Пользователь может узнать значение, кото		
Определение строки с сообщением об ошибке. <code>error_message</code> – это метка, которая обозначает начало определения строки. <code>DB</code> – директива ассемблера, которая означает "define byte" (определить байт). Она используется для определения данных в виде байт. “Error: Invalid N or j values!” — это сама строка, которая будет выведена как сообщение об ошибке. Строка заключена в одинарные кавычки, что означает, что она рассматривается как последовательность байт. <code>0Dh</code> — это код ASCII для символа перевода строки (newline). <code>0Ah</code> — это код ASCII для символа возврата каретки (carriage return). <code>\$</code> — это специальный символ, который обозначает конец строки. Он часто используется в ассемблере для обозначения конца строки.	<code>error_message</code>	DB (define byte)
Определение строки с сообщением о переполнении. Эта строка определяется аналогично первой, но содержит сообщение о переполнении: “ Error: Overflow occured!”	<code>overflow_message</code>	DB (define byte)

Кодирование

Этап кодирования позволяет понять механизм работы программы, поскольку здесь происходит описание команд ассемблеру, в кодировании важен порядок исполнения команд, и может произойти ситуация, когда одна и та же команда описывается более одного раза, так как это требует этап кодирования программы.

Описание кодирования:

- Начало программы:

CODE SEGMENT – начало сегмента кода программы.

ASSUME CS:CODE – указывает, что сегмент кода является сегментом по умолчанию.

ORG 100H – устанавливает начало программы по смещению 100h.

- Проверка значений N и j:

MOV AX, [N] – команда MOV загружает значение N в регистр AX.

CMP AX, 0 – команда сравнения CMP вычитает ноль из значения AX, таким образом сравнивая значение AX с нулем: если оно меньше нуля, то устанавливает флаг SF (Sign Flag – флаг состояний) в единицу, переходит к метке «error_main».

JS error_main – команда условного перехода JS: если команда сравнения CMP (CMP AX, 0) установила SF в единицу, то происходит переход к метке «error_main».

CMP AX, 1 – команда CMP вычитает единицу из значения AX, таким образом сравнивая значение AX с единицей: если значение AX больше единицы, то устанавливает флаги CF и ZF (Zero Flag – флаг нуля) в ноль и ноль соответственно.

JA NEXT – команда условного перехода JA (для беззнаковых чисел): если команда сравнения CMP (CMP AX, 1) установила флаги CF (CF – с помощью этого флага можно понять, что вычитание произошло без заема из старшего разряда в случаях, если происходит вычитание большего числа из меньшего) и

ZF (ZF – значение результата команды CMP не является нулем) в нули, таким образом, переход произойдет по метке «NEXT», если значение AX больше нуля, то есть при сравнении командой CMP через вычитание единицы из значения AX, которое произошло без заема из старшей половинки, а значит результат этого вычитания больше или равен числу, которое вычиталось из значения AX (в данном случае, – это единица), это был флаг CF, проверяющий отношение “больше или равно”, но проверяется еще и флаг ZF, если он оказался установлен в ноль, то значение AX строго больше единицы. Поскольку если бы оно было равно единице, то флаг ZF установился бы в единицу, потому что результат сравнения командой CMP был бы равен нулю.

JE error_main – команда условного перехода JE (для беззнаковых чисел): если команда сравнения CMP (CMP AX, 1) установила флаг ZF в единицу, то это значит, что результат вычитания единицы из значения AX равен нулю, потому что они равны, а значит происходит переход к метке «error_main».

CMP AX, 0 – команда CMP вычитает ноль из значения AX, сравнивая значение AX с 0: если значение AX равно нулю, – команда установит флаг ZF в единицу, поскольку значение AX равно нулю.

JE error_main – команда условного перехода JE (для беззнаковых чисел): если команда сравнения CMP (CMP AX, 0) установила флаг ZF в единицу, то это значит, что результат вычитания нуля из значения AX равен нулю, потому что они равны, а значит происходит переход к метке «error_main».

NEXT: – метка «NEXT», переход на метку совершается, чтобы избежать избыточных проверок (JA NEXT – условный переход совершается в случае, если значение AX больше единицы, поэтому нет нужды в том, чтобы проверять значение AX на равенство нулю).

MOV AX, [j] – команда MOV загружает значение j в регистр AX.

CMP AX, 1 – команда CMP вычитает единицу из значения AX, таким образом сравнивая значение AX с единицей: если значение AX равно единице, то флаг ZF устанавливается в единицу.

JE error_main – команда условного перехода JE (для беззнаковых чисел): если команда сравнения CMP (CMP AX, 1) установила флаг ZF в единицу, то это значит, что результат вычитания единицы из значения AX равен нулю, потому что они равны, а значит происходит переход к метке «error_main».

CMP AX, 0 – команда CMP вычитает единицу из значения AX, таким образом сравнивая значение AX с единицей: если значение AX равно единице, то флаг ZF устанавливается в единицу.

JE error_main – команда условного перехода JE (для беззнаковых чисел): если команда сравнения CMP (CMP AX, 0) установила флаг ZF в единицу, то это значит, что результат вычитания нуля из значения AX равен нулю, потому что они равны, а значит происходит переход к метке «error_main».

- Установка обработчика прерывания:

MOV AH, 25h – команда MOV загружает 25h в регистр AH (функция DOS для установки вектора прерывания).

MOV AL, 60h – загружает 60h в регистр AL (номер прерывания INT 60h).

LEA DX, ISR – загружает в регистр DX адрес подпрограммы ISR.

INT 21h – вызывает прерывание DOS (INT 21h).

- Вызов прерывания:

INT 60h – вызывает прерывание INT 60h.

- MOV [result], AX – команда MOV сохраняет значение AX (результат) в ячейку памяти по адресу «result».

- Восстановление старого обработчика прерывания:

MOV AH, 35h – загружает 35h в регистр AH (функция DOS для получения вектора прерывания).

MOV AL, 60h – загружает 60h в регистр AL (номер прерывания INT 60h).

LEA DX, OLD_ISR – загружает в регистр DX адрес подпрограммы OLD_ISR.

INT 21h – вызывает прерывание DOS (INT 21h).

- Завершение программы:

MOV AX, 4Ch – загружает 4Ch в AX (код функции завершения программы).

INT 21h – вызывает прерывание DOS (INT 21h).

- Обработка ошибок:

error_main: – метка «error_main».

MOV DX, offset error_message – загружает в регистр DX адрес сообщения об ошибке с названием «error_message»

MOV AH, 09h – загружает 09h в регистр AH (функция DOS для вывода строки).

INT 21h – вызывает прерывание DOS (INT 21h).

MOV AX, 4Ch – загружает 4Ch в AX (код функции завершения программы).

INT 21h – вызывает прерывание DOS (INT 21h).

- Прерывание ISR:

ISR PROC FAR – начало процедуры «ISR» (обработчик прерывания).

Вычисление выражения:

MOV AX, [N] – команда MOV загружает значение N в регистр AX.

MOV BX, [j] – команда MOV загружает значение j в регистр BX.

MOV CX, AX – команда MOV загружает значение AX (N) в регистр CX (счетчик цикла).

MOV SI, 1 – команда MOV загружает единицу в регистр SI (i = 1).

MOV BP, 1 – команда MOV загружает единицу в регистр BP (результат произведения)

MOV DI, BX – команда MOV загружает значение BX (j) в регистр DI.

loop_i: – метка начала цикла с названием «loop_i».

MOV AX, SI – команда MOV загружает значение SI (i) в регистр AX.

MOV BX, DI – команда MOV загружает значение DI (j) в регистр BX.

IMUL SI – команда IMUL (произведение для чисел со знаком) вычисляет SI * SI и сохраняет результат в SI (i = i * i).

IMUL SI – команда IMUL вычисляет SI * SI и сохраняет результат в SI (i = i * i * i).

IMUL BX – команда IMUL вычисляет SI * BX и сохраняет результат в SI (i = i * i * i * j).

MOV BX, BP – команда MOV загружает значение BP (текущий результат) в регистр BX.

IMUL BX – команда IMUL умножает AX на i^3*j , результат в DX:AX.

JC overflow – если установлен флаг переноса (переполнение), переходит к метке «overflow».

MOV BP, AX – команда MOV загружает значение AX (младшее слово результата) в регистр BP.

INC SI – команда INC (инкремент) увеличивает значение в SI на единицу (i++).

LOOP loop_i – уменьшает CX на единицу и переходит к метке «loop_i», если CX не равно нулю.

- Сохранение результата:

MOV AX, BP – команда MOV загружает значение BP (результат) в регистр AX.

JMP end_isr – команда безусловного перехода совершает переход к метке «end_isr».

- Обработка переполнения:

overflow: – метка «overflow».

MOV DX, offset overflow_message – загружает в регистр DX адрес сообщения о переполнении, которое имеет название «overflow_message».

MOV AH, 09h – загружает 09h в регистр AH (функция DOS для вывода строки).

INT 21h – вызывает прерывание DOS (INT 21h).

MOV AX, 0 – загружает ноль в регистр AX.

JMP end_isr – команда безусловного перехода совершает переход к метке «end_isr».

- Процедура (метка) «end_isr»:

end_isr: – метка «end_isr».

IRET – выход из прерывания.

- Старый обработчик прерывания:

OLD_ISR PROC FAR – начало процедуры «OLD_ISR».

IRET – выход из прерывания.

OLD_ISR ENDP – конец процедуры «OLD_ISR».

- Определение переменных:

N DW 3 – определяет переменную N как слово (2 байта), значение = 3.

j DW 3 – определяет переменную j как слово (2 байта), значение = 3.

result DW 0 – определяет переменную result как слово (2 байта), значение = 0.

error_message DB 'Error: Invalid N or j values!', 0Dh, 0Ah, '\$' – определяет строку с сообщением об ошибке.

overflow_message DB 'Error: Overflow occurred!', 0Dh, 0Ah, '\$' – определяет строку с сообщением о переполнении.

- Конец программы:

CODE ENDS – конец сегмента кода.

END Start – конец программы.

Тестирование программы

Дата проведения тестирования: 20.11.24.

Исходные данные значений параметров N и j, результат работы программы при этих входных данных, а также результат тестирования можно представить в виде табл. 2:

Табл. 2

Тестирование программы при различных значениях параметров N и j

Параметр N	Параметр j	Результат работы программы	Результат тестирования
3	3	Подпрограмма вызвалась через вызов пользовательского прерывания (INT 60h). Адрес подпрограммы был занесен в таблицу векторов прерываний при помощи функций DOS 25h и 60h. Подпрограмма вернула значение 5832 в регистре AX, чтобы отобразить этот результат; оно было загружено в ячейку памяти с именем «result» после выхода из прерывания 60h. Далее происходит восстановление программой адреса старого обработчика прерывания при помощи функций DOS 35h и 60h. Происходит завершение программы: в регистр AX загружается значение 4Ch (код функции завершения программы), затем вызывается прерывание DOS (INT 21h).	Успех
0	3	Поскольку N не больше единицы, то подпрограмма не вызвалась, следовательно, произошел переход по метке «error_main», где в регистр DX был загружен адрес сообщения об ошибке, а в регистр AH было загружено значение 09h, – это функция DOS для вывода	Успех

		строки. Затем вызвалось прерывание DOS (INT 21h), чтобы вывести сообщение об ошибке на экран («Error: Invalid N or j values!»), затем в регистр AX было загружено значение 4Ch – это код функции завершения программы, затем вызвалось прерывание DOS (INT 21h).	
10	10	Поскольку значения параметров N и j оказались нетривиальны, и N больше единицы, то подпрограмма запустилась, однако в процессе выполнения подпрограммой вычислений при использовании команды IMUL произошло переполнение, то есть результат арифметической операции умножения со знаком не помещается в регистр AX, следовательно, происходит условный переход JC на метку «overflow», где происходит обработка переполнения с выводом сообщения на экран об ошибке («Error: Overflow occurred!"), в регистр DX загружается значение адреса сообщения о переполнении с именем «overflow_message». Затем в регистр AH загружается значение 09h, – это функция DOS для вывода строки. Далее вызывается прерывание DOS (INT 21h). Происходит обнуление регистра AX и безусловный переход к метке «end_isr», в которой происходит выход из прерывания. Происходит возвращение к месту вызова пользовательского прерывания 60h. Далее происходит восстановление программой адреса старого обработчика прерывания при помощи функций DOS 35h и 60h. Происходит завершение программы: в регистр AX загружается значение 4Ch (код	Успех

		функции завершения программы), затем вызывается прерывание DOS (INT 21h).	
1	1	Поскольку хотя бы один из двух параметров N и j содержит тривиальное значение (единица), то подпрограмма не вызвалась, следовательно, произошел переход по метке «error_main», где в регистр DX был загружен адрес сообщения об ошибке, а в регистр AH было загружено значение 09h, – это функция DOS для вывода строки. Затем вызвалось прерывание DOS (INT 21h), чтобы вывести сообщение об ошибке на экран, затем в регистр AX было загружено значение 4Ch – это код функции завершения программы, затем вызвалось прерывание DOS (INT 21h).	Успех
3	-2	Подпрограмма вызвалась через вызов пользовательского прерывания (INT 60h). Адрес подпрограммы был занесен в таблицу векторов прерываний при помощи функций DOS 25h и 60h. Подпрограмма вернула значение -1728 в регистре AX, чтобы отобразить этот результат; оно было загружено в ячейку памяти с именем «result» после выхода из прерывания 60h. Далее происходит восстановление программой адреса старого обработчика прерывания при помощи функций DOS 35h и 60h. Происходит завершение программы: в регистр AX загружается значение 4Ch (код функции завершения программы), затем вызывается прерывание DOS (INT 21h).	Успех

-2	2	Поскольку N не больше единицы, то подпрограмма не вызвалась, следовательно, произошел переход по метке «error_main», где в регистр DX был загружен адрес сообщения об ошибке, а в регистр AH было загружено значение 09h, – это функция DOS для вывода строки. Затем вызвалось прерывание DOS (INT 21h), чтобы вывести сообщение об ошибке на экран («Error: Invalid N or j values!»), затем в регистр AX было загружено значение 4Ch – это код функции завершения программы, затем вызвалось прерывание DOS (INT 21h).	
0	0	Поскольку хотя бы один из двух параметров N и j содержит тривиальное значение (ноль), то подпрограмма не вызвалась, следовательно, произошел переход по метке «error_main», где в регистр DX был загружен адрес сообщения об ошибке, а в регистр AH было загружено значение 09h, – это функция DOS для вывода строки. Затем вызвалось прерывание DOS (INT 21h), чтобы вывести сообщение об ошибке на экран, затем в регистр AX было загружено значение 4Ch – это код функции завершения программы, затем вызвалось прерывание DOS (INT 21h).	

Вывод по проведению тестирования программы:

Программа успешно прошла тестирование со значениями параметров, не вызывающими переполнение. Подпрограмма обработки прерывания функционирует правильно, вычисления производятся корректно, результат возвращается. Программа завершается успешно.

Заключение

Подводя итоги курсовой работы, были приобретены навыки написания программы на ассемблере, которая вызывает подпрограмму с параметрами, которая после своего выполнения возвращает результат в ячейку памяти result (результат вычисления). Были приобретены навыки работы с функциями DOS 25h, 35h и 60h. Был получен опыт вызова пользовательского прерывания 60h. С помощью функции 25h был получен опыт занесения адреса подпрограммы в таблицу векторов прерываний, а пользовательское прерывание оказалось необходимым, поскольку остальная программа не должна зависеть от выполнения одной подпрограммы, поскольку язык ассемблер создан для аппаратных вычислений, которые длятся быстро, но чтобы они выполнялись синхронно, не завися друг от друга, нужно выполнять пользовательское прерывание INT 60h. В результате использования 35h был получен опыт восстановления адреса старого обработчика прерывания. Также было отмечено как факт, что перед вызовом пользовательского прерывания нужно выбрать регистр общего назначения, который имеет 8-битные половинки (например, AX=AH:AL), то в регистр AH заносится название функции DOS, а в регистр AL – номер прерывания, в этом случае – пользовательского.

Список литературы

1. Системное программирование [Электронный ресурс]
<https://bigenc.ru/c/sistemnoe-programmirovanie-602649>. Дата обращения:
19.01.2025
2. Л.Н. Виноградова. Указания к выполнению лабораторных работ по дисциплине «системное программирование»: Команды условных и безусловных переходов. Учебно-методическое пособие. [Электронный ресурс] – Череповец: ФГБОУ ВПО «Череповецкий государственный университет»: Институт информационных технологий 2013. – 35 с.

Приложение 1. Техническое задание

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ЧЕРЕПОВЕЦКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт информационных технологий

наименование института (факультета)

Математическое и программное обеспечение ЭВМ

наименование кафедры

Программирование на ассемблере

наименование дисциплины в соответствии с учебным планом

УТВЕРЖДАЮ

Зав. кафедрой _____,

д.т.н., профессор _____ Ершов Е.В.

«___» _____ 20__ г.

ТЕМА КУРСОВОЙ РАБОТЫ

Программирование на языке низкого уровня

Техническое задание на курсовую работу

Листов 16

Руководитель Виноградова Людмила

Николаевна

Ф.И.О преподавателя

Исполнитель

студент 1ПИБ-02-3оп-23

группа

Бекошин Никита Вячеславович

Фамилия, имя, отчество

2024 год

Введение

Данная курсовая работа посвящена разработке программы для расчета математического выражения на низком уровне. Целью курсовой работы является разработка программы на языке ассемблера, демонстрирующая работу с прерываниями, регистрами и памятью для вычисления заданного математического выражения. Основными требованиями являются реализация перехвата прерывания и обработчика прерывания.

Перехват прерывания означает, что программа должна перехватывать вектор прерывания 60h, чтобы установить собственный обработчик.

Обработчик прерывания называется таковым, потому что при вызове прерывания 60h должен выполняться следующий сценарий: сохранение значений регистров общего назначения в стеке, затем вызов подпрограммы расчета математического выражения, передавая входные значения N и j через регистры общего назначения. После выполнения подпрограммы должна производиться запись результата вычисления в выделенную ячейку памяти, затем восстановление значений регистров общего назначения из стека и возврат к вызывающей программе.

- Подпрограмма расчета:

Подпрограмма должна принимать значения N и j через регистры общего назначения.

Подпрограмма должна выполнять расчет заданного математического выражения (рис. 1), используя значения N и j.

$$\prod_{i=1}^N i^{3 \cdot j}$$

Рис. 1. Математическое выражение

Результат расчета должен быть возвращен и записан в выделенную ячейку памяти.

- Вывод результата:

После возврата из прерывания программа должна вызвать подпрограмму, которая выводит сохраненный результат из ячейки памяти на экран.

Восстановление вектора прерывания:

В конце работы программы должен быть восстановлен исходный вектор прерывания 60h.

- Язык реализации программы:

Программа должна быть реализована на языке ассемблера.

1. Основания для разработки

Основанием для разработки приложения является задание на курсовую работу по дисциплине "Программирование на ассемблере", выданное на кафедре МПО ЭВМ ИИТ ЧГУ.

Наименование темы разработки: "Программирование на языке низкого уровня".

2. Назначение разработки

Разработка программы направлена на развитие навыков и умений при работе с памятью, эта деятельность обеспечивается низкоуровневым доступом на уровне процессора, и целью разработки программы является написание подпрограммы, выполняющей определенные действия в памяти процессора, если говорить конкретнее, - она должна вычислять значение произведения на уровне процессора, что помогает напрямую контролировать распределение ресурсов для подсчета этого значения и, следовательно, эффективного использования памяти, чтобы сохранить максимально возможный уровень производительности компьютера.

3. Требования к программе

3.1. Требования к функциональным характеристикам

Требуется написать программу, которая вычисляет значение выражения под номером 6.

6. Вычислить значение произведения: $\prod_{i=1}^N i^3 \cdot j$

Подпрограмма должна:

- выполнять действия, указанные в конкретном задании (под номером 6), при этом подпрограмме должны передаваться параметры N и j;
- выполняться через вызов пользовательского прерывания (например, INT 60h). Адрес подпрограммы должен быть занесен в таблицу векторов прерываний при помощи функций DOS 25h и 35h;
- принимать значения параметров N и j только при условии, если эти значения нетривиальны, то есть ни один из двух параметров не содержит ноль или единицу в качестве значения. Значение параметра N должно быть больше единицы;
- проверять работу программы для нескольких пар значений параметров N и j.

Подпрограмма должна возвращать результаты работы в регистрах общего назначения. После вызова подпрограммы программа должна восстановить адрес старого обработчика прерывания при помощи тех же функций DOS (25h и 35h). Подпрограмма должна принимать параметры N и j, как только они будут передаваться в неё. Параметры N и j могут передаваться в подпрограмму обработки прерывания через регистры общего назначения или через ячейки памяти.

Подпрограмма должна учитывать ситуацию переполнения разрядной сетки для факториальных выражений при вычислении произведений значений N и j.

Подпрограмма должна учитывать приоритеты арифметических операций при вычислении значений сумм и произведений с использованием арифметических операций.

Подпрограмма должна выполнять команду INT в режиме пошагового выполнения команды (выполнение активируется нажатием комбинации клавиш Alt+F7), чтобы подпрограмма могла выполнять обработку прерывания в процессе проверки работы программы в отладчике.

3.2. Требования к надежности

В ходе написания программы нужно понимать, что запуск программы при определенных значениях входных данных не должен приводить к утечке данных. Кроме требований тривиальности (значения параметров N и j не равны нулю или единице, N больше единицы, j может принимать любое другое значение, кроме нуля и единицы, но учитывая особенности ассемблера, значения N и j должны быть целочисленными), чтобы не происходило утечек, в подпрограмме реализован алгоритм проверки после произведения на случай переполнения ячейки памяти регистра AX, поэтому потребуется использовать условный переход JC на метку с названием «overflow», в которой происходит обнуление регистра AX, чтобы избежать утечки, затем вывести сообщение на экран о том, что произошло переполнение, и затем выйти из подпрограммы, чтобы Пользователь повторил попытку запуска программы, но уже при других значениях параметров N и j.

3.3. Условия эксплуатации

Все авторские права на продукт принадлежат производителю, разработчику приложения.

Запрещается использовать ПО для создания или распространения вредоносного ПО.

Гарантия не распространяется на повреждения, вызванные неправильным использованием или модификацией программы.

3.4. Требования к составу и параметрам технических средств

Программа не требует высокой производительности персонального компьютера пользователя, однако должны быть соблюдены минимальные системные требования для Windows 10:

- Процессор: не менее 1 ГГц или SoC (System on a Chip - в русской аббревиатуре СнК – система на кристалле).
- Оперативное запоминающее устройство (ОЗУ): 1 ГБ для 32-разрядной операционной системы (ОС) или 20 ГБ для 64 разрядной ОС.
- Место на жестком диске: 16 ГБ для 32-разрядной ОС или 20 ГБ для 64-разрядной ОС.
- Видеоадаптер DirectX 9 или более поздняя версия с драйвером WDDM 1.0.
- Экран: 800х600.

3.5. Требования к информационной и программной совместимости

Программу можно запустить на компьютере или ноутбуке, на котором установлена операционная система Windows.

3.6. Требования к маркировке и упаковке

Программа как продукт может использоваться другими пользователями бесплатно, но основное предназначение программы – создание опоры для дальнейшего развития в программировании на языке низкого уровня, поскольку программа – это основа, с помощью которой будут писаться программы на ассемблере.

3.7. Требования к транспортированию и хранению

Предполагается, что программу можно хранить на компакт-диске, и даже не важно, будут ли дорабатываться функциональные возможности программы или нет, потому что данный вид хранения позволяет избежать потери, утечек или даже уничтожения проекта.

3.8. Специальные требования

Требования к вызову подпрограммы включают в себя возможность программы автоматически восстанавливать адрес старого обработчика прерывания при помощи функций DOS (25h и 35h).

Требования к параметрам включают в себя возможность подпрограммы, которая проводит обработку прерывания, принимать параметры N и j как только они будут передаваться в неё через регистры общего назначения или через ячейки памяти.

Требования к учету подпрограммой ситуации переполнения разрядной сетки для факториальных выражений в случаях вычислений произведений значений N и j .

Требования к учету подпрограммой приоритетов арифметических операций (умножение, деление, вычитание, сложение) при вычислении значений сумм и произведений с использованием этих операций, в том числе операции возведения в степень « $\wedge$ », в случае её неявного указания, она выполняется только для значения, перед которым стоит эта операция, а значение, которое стоит после операции « $\wedge$ », является показателем степени, если правее от показателя степени имеются еще арифметические операции с другими значениями, то они не относятся к показателю степени значения, к которому применяется операция « $\wedge$ ». Явное указание применимости арифметической операции « $\wedge$ » к более чем одному значению, при условии, что между этими значениями находятся какие-нибудь арифметические операции, достигается за счет скобок «(» и «)», которые означают левую и правую границы проведения арифметического расчета

значений, между которыми установлены арифметические операции. Эти скобки допустимо использовать как в основании степени, так и в её показателе.

Требования к проведению обработки прерывания в реальном времени включают в себя возможность подпрограммы выполнять команду INT в режиме пошагового выполнения команды.

4. Требования к программной документации

4.1. Содержание расчётно-пояснительной записки

Программная документация должна содержать расчётно-пояснительную записку с содержанием:

- Титульный лист.
- Оглавление.
- Введение.
- Описание предметной области.
- Постановка задачи.
- Логическое проектирование.
- Физическое проектирование.
- Кодирование.
- Тестирование программы.
- Заключение.
- Список литературы.
- Приложение 1. Техническое задание.
- Приложение 2. Руководство пользователя.
- Приложение 3. Текст программы.
- Приложение 4. Листинг программы.

4.2. Требования к оформлению

Требования к оформлению, установленные государственным стандартом (ГОСТ), должны быть соблюдены в процессе работы без возможности пропуска оформления отдельных компонентов документа. (в табл. П1.1 – все требования, которые должны быть выполнены)

Таблица П1.1

Требования к оформлению

Документ	Печать на отдельных листах формата А4 (210х297 мм); оборотная сторона не заполняется; листы нумеруются. Печать возможна ч/б. Файлы предъявляются на компакт-диске: РПЗ с ТЗ; программный код. Листы и диск в конверте вложены в пластиковую папку скоросшивателя.
Страницы	Ориентация – книжная; отдельные страницы, при необходимости, альбомная. Поля: верхнее, нижнее – по 2 см, левое – 3 см, правое – 1 см.
Абзацы	Межстрочный интервал – 1,5, перед и после абзаца – 0.
Шрифты	Кегль – 14. В таблицах шрифт 12. Шрифт листинга – 10 (возможно в 2 колонки).
Рисунки	Подписывается под ним по центру: Рис.Х. Название В приложениях: Рис.П1.3. Название.
Таблицы	Подписывается: над таблицей, выравнивание по правому: «Таблица Х». В следующей строке по центру Название Надписи в «шапке» (имена столбцов, полей) – по центру. В теле таблицы (записи) текстовые значения – выровнены по левому краю, числа, даты – по правому.

Общие требования к тексту	Красная строка. Выравнивание по ширине, в т.ч. в таблицах. Нумерация страниц. Титульная страница – первая, не нумеруется. Использование распространенных сокращений: рис.1, табл.2, прил.3.
---------------------------	---

5.Стадии и этапы разработки

Создание программы, которая вычисляет значение выражения под номером 6 будет распланировано по следующей траектории (табл. П1.2).

Таблица П1.2

Стадии и этапы разработки

Наименование этапа разработки	Сроки разработки	Результат выполнения	Отметка о выполнении
Определение варианта задания из 1 части	22.10.2024	Определен вариант задания под номером 6.	
Написание программы и подпрограммы	15.11.2024	Написана программа и подпрограмма	
Тестирование	20.11.2024	Подпрограмма вызывалась через вызов пользовательского прерывания (INT 60h). Адрес подпрограммы был занесен в таблицу векторов прерываний при помощи функций DOS 25h и 35h. Подпрограмма выполняет действия, указанные в конкретном задании порядка. Программа передает Подпрограмме параметры N ($N > 1$) и j ($j \neq 0, j \neq 1$) через регистры общего назначения	

		(AX,CX,BX,DX) или через ячейки памяти. Вычисляются произведения значений параметров N и j, в качестве N и j подобраны такие значения, при которых не будет происходить ситуации переполнения разрядной сетки для факториальных выражений. Подпрограмма возвращает результат вычисления в регистрах общего назначения. Для вычисления значений сумм и произведений с помощью арифметических операций. Так как приоритеты арифметических операций учитываются, то вычисления значений сумм и произведений происходят успешно. Происходит восстановление программой адреса старого обработчика прерывания при помощи функций DOS.	
--	--	--	--

6. Порядок контроля и приемки

Данный пункт показывает демонстрацию порядка выполнения всей курсовой работы, связанной с написанием программы, которая вычисляет значение выражения из задания первой части №6 и с соответствующей документацией (табл. П1.3).

Порядок контроля и приемки

Наименование этапа разработки	Сроки разработки	Результат выполнения	Отметка о выполнении
Оформление технического задания.	22.10.2024	Документ “техническое задание”.	
Написание программы и подпрограммы	12.12.2024	Программа и подпрограмма	
Составление РПЗ.	12.12.2024	РПЗ со всеми имеющимися пунктами документации в полностью оформленном виде.	
Сдача РПЗ и оценка.	16.12.2024	Получение оценки как итогового результата курсовой работы.	

Приложение 2. Руководство пользователя

Когда Пользователь установит входные данные в ячейки памяти, которые предназначены для хранения значений параметров N и j , а также подготовит ячейку памяти под абсолютный результат вычислений (не промежуточный результат, а итоговый) (рис. 1).

```
N DW 3
j DW 3
result DW 0
```

Рис. 1. Входные данные для проведения вычислений подпрограммой итогового значения произведения выражений, согласно заданию курсовой работы

Если Пользователь скомпилирует программу, затем запустит её, то он сможет увидеть, какой имеется итоговый результат общего вычисления, который произвела подпрограмма, которая впоследствии вернула значения в преобразованном виде, то есть получила исходные данные, сделала вычисления, затем вернула результат после выполнения своей задачи (рис 2).

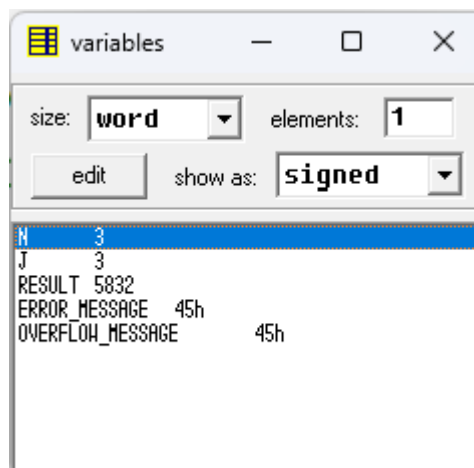


Рис. 2. Результат отображен в ячейке памяти переменной result: 5832 при входных данных значений параметров N и j : 3 и 3 соответственно

Если Пользователь установит значения параметров N и j как тривиальные, то есть когда хотя бы один из них будет равно единице или нулю (рис. 3), то программа выведет соответствующее сообщение, предупреждающее об ошибке (рис. 4):

```
N DW 3
j DW 1
```

Рис. 3. Пример, когда хотя бы один из двух параметров (N и j) тривиален

SCM emulator screen (80x25 chars)

```
Error: Invalid N or j values!
```

Рис. 4. Вывод сообщения об ошибке тривиальности на экран

Может возникнуть ситуация переполнения разрядной сетки для факториальных выражений при вычислении произведений значений N и j. Например, если ввести значения параметров N и j, как указано на рис. 5, то на рис. 6 можно заметить сообщение, предупреждающее о том, что в процессе вычисления произведения его результат не поместился в регистре AX, следовательно значение регистра AX обнуляется.

```
N DW 4
j DW 4
```

Рис. 5. Установленные Пользователем значения параметров N и j в соответствующих их именам ячейках памяти

SCM emulator screen (286x66 chars)

```
Error: Overflow occurred!
```

Рис. 6. Вывод сообщения об ошибке переполнения на экран

Может возникнуть ситуация, что если Пользователь в качестве N введет отрицательное число (рис. 7), то программа также выведет сообщение на экран об ошибке (рис. 8), потому что кроме того, что значения N, как и j, должно быть нетривиально, но для N особое условие – его значение должно быть больше единицы, что говорит о том, что оно не должно быть отрицательное.

```
N DW -1
j DW 3
```

Рис. 7. Ячейка памяти N установлена Пользователем как значение -1



Рис. 8. Вывод сообщения об ошибке, потому что значение N оказалось не больше, чем единица, на экран

Программа допускает, что Пользователь инициализирует значение параметра j отрицательным числом (рис 9), следовательно, программа не выведет сообщение об ошибке на экран

```
N DW 3
j DW -2
```

Рис. 9. Ячейка памяти j установлена Пользователем как значение -2 (отрицательное число)

Приложение 3. Текст программы

```
CODE SEGMENT
ASSUME CS:CODE
ORG 100H
```

```
Start:
```

```
MOV AX, [N]
CMP AX, 0
JS error_main
CMP AX, 1
JA NEXT
JE error_main
```

```
CMP AX, 0
JE error_main
NEXT:
MOV AX, [j]
CMP AX, 1
JE error_main
CMP AX, 0
JE error_main
```

```
MOV AH, 25h
MOV AL, 60h
LEA DX, ISR
INT 21h
```

```
INT 60h
MOV result, AX
```

```
MOV AH, 35h
MOV AL, 60h
LEA DX, OLD_ISR
INT 21h
```

```
MOV AX, 4CH
INT 21H
```

```
error_main:
MOV DX, offset error_message
MOV AH, 09h
INT 21h
MOV AX, 4CH
INT 21H
```

```
ISR PROC FAR
```

```
MOV AX, [N]
MOV BX, [j]
```

```
MOV CX, AX
MOV SI, 1
MOV BP, 1
```

```
MOV DI, BX
```

```
loop_i:
```

```
MOV AX, SI
MOV BX, DI
IMUL SI
IMUL SI
IMUL BX
```

```
MOV BX, BP
IMUL BX
```

```
JC overflow
MOV BP, AX
INC SI
LOOP loop_i
MOV AX, BP
JMP end_isr
```

```
overflow:
```

```
MOV DX, offset overflow_message
MOV AH, 09h
INT 21h
MOV AX, 0
JMP end_isr
```

```
end_isr:
```

```
IRET
```

```
error:
```

```
MOV AX, 0
IRET
```

```
ISR ENDP
```

```
OLD_ISR PROC FAR
```

```
IRET
```

```
OLD_ISR ENDP
```

```
N DW 3
```

```
j DW 3
```

```
result DW 0
```

```
error_message DB 'Error: Invalid N or j values!', 0Dh,
0Ah, '$'
overflow_message DB 'Error: Overflow occurred!',
0Dh, 0Ah, '$'
```

```
CODE ENDS
```

```
END
```

```
Start
```

Приложение 4. Листинг программы